

Coding & Solution

Date	01 July 2026
Team ID	XXXX
Project Name	Rising Waters
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the quality and completeness of the implemented code and the overall solution for the Rising Waters flood prediction system. The submission demonstrates working functionality, clean code practices, and alignment with the defined requirements.

Instructions:

1. Ensure your code is well-structured, commented, and follows best practices for your chosen technology stack.
2. The solution should address the core problem statement and implement the key functional requirements.
3. Include all source files, configuration files, and setup instructions needed to run the application.
4. Attach or link to your code repository (e.g., GitHub) in the table below.

Solution Summary

Field	Details
Repository Link / URL	https://github.com/Gayatri344/smart--bridge_Rising-Water
Programming Language(s)	Python, HTML, CSS, JavaScript
Framework(s) Used	Flask, Scikit-learn, XGBoost, Pandas, NumPy, Matplotlib, Seaborn
Key Features Implemented	Data preprocessing & EDA (univariate/multivariate analysis, box plots, heat maps); training and comparison of Decision Tree, Random Forest, KNN, and XGBoost classifiers; best model (XGBoost, 96.55% test accuracy) saved as floods.save; Flask web app with Home Page, Prediction Input Page, and Flood Chance / No Flood Chance Result Pages for real-time flood risk prediction.
Pending / Incomplete Features	Live deployment to IBM Cloud; prediction history / logging dashboard; user authentication for authorities/coordinators; automated unit tests for the Flask routes.
Setup / Run Instructions	1) Install Python 3.8+ and required libraries (numpy, pandas, scikit-learn, matplotlib, seaborn, flask, joblib). 2) Place the dataset and trained model file (floods.save) in the project directory. 3) Run 'python app.py' to start the Flask server. 4) Open the local URL in a browser, enter rainfall and weather inputs on the Prediction Input Page, and view the flood risk result.

Code Quality Checklist

S.No	Criteria	Status (Yes/No)
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes

Additional Notes / Comments:

The Rising Waters codebase follows a clean, modular structure separating data preprocessing, model training/evaluation, and the Flask web application. The XGBoost model was selected as the best performer (96.55% accuracy) and integrated into an intuitive prediction interface. Remaining work includes finalizing the live IBM Cloud deployment and adding automated tests.